Gestir: Reading and Writing Maya's Binary Scene Format Without Maya

Rigel Bowen Version 1.0, August 2026 Contact: fact@rigelisawesome.com

---

1. Abstract

Gestir is a dependency-free C++ reader and writer for Maya's binary scene format, the `.mb` file. It doesn't link against the Maya SDK, OpenMaya, or any Autodesk library, and it doesn't need a Maya installation, a license seat, or a running DCC of any kind. It reads the file off disk, walks the container, and hands back structured scene data: meshes with topology and UV sets, transform hierarchies with pivots, per-face shading assignments, cameras, animation curves with real keyframe times, and the connection graph wiring them together. It also writes, performing structural edits back into an existing scene file.

The writer's central claim is about the chunk-tree model rather than about copying bytes. Re-emitting an unmodified file reproduces it byte for byte because every chunk's offset, span, and trailing pad is reconstructed from the parsed model, so a byte-identical result is evidence the model is a faithful description of the file, not evidence that a buffer was memcpy'd.

None of it came from a specification. Autodesk doesn't publish the `.mb` layout, and the format carries no self-describing schema past four-character chunk tags. Everything here came out of differential analysis: hexdumps, chunk walkers, controlled saves diffed against each other, and Maya itself as the final say on whether a written file is valid: if Maya opens it clean, it's valid, and if it doesn't, it isn't. The name is Old Norse for "guests", which sounds wrong until you know the Gestir were the corps in the Norwegian king's retinue who went out ahead and reported back.

Provenance. Every finding in this paper comes from analyzing files the application itself wrote. I did not decompile or disassemble any Autodesk binary, I did not read SDK headers to recover format layout, and I did not touch any licensing mechanism. The method throughout is: save a file, look at the bytes, form a hypothesis, save another file, check. The purpose is interoperability, specifically getting data into and out of my own scenes without a running copy of the application.

This paper covers three things. The container as I observe it, including a correction to the assumption I made that cost me the most time. The reverse-engineering methodology, which is the part that generalizes to any undocumented binary format. And the writer architecture, "verbatim unless dirty", which is what makes it safe to modify a file whose format you only partly understand, plus the one bug that got past it.

Gestir is the native layer under Yggdrasil, my Houdini-Maya destruction round-trip pipeline. It's been in daily use on production-scale scenes since spring 2026.

2. The problem, and what falls out of solving it

Every pipeline question that touches a Maya scene has one expensive default answer: open the scene in Maya and ask. That costs a full application startup, a license checkout, and however long the scene takes to load, and it costs all of it again for the next file. `mayapy` doesn't fix that. It's the same runtime with a smaller face on it and it still wants a license.

Fine with one scene and a human in front of it. It falls apart at scale. "Which of these four hundred assets have more than one UV set?" is a five-second question against the bytes and a two-hour question against Maya. "Which shading group owns which faces on this mesh, in a file nobody has opened?" is worse, because the obvious approach loads the scene, resolves every reference underneath it, and evaluates a graph I didn't care about, all to read a partition sitting in the file as a list of integer pairs.

The return leg is the other half. My pipeline simulates destruction in Houdini and has to put the result back into Maya scenes: transforms, per-frame animation, shading assignments that survived a fracture. Through Maya that means a human opening a file, running a tool, and saving. Against the bytes it means a batch job.

The requirements were narrow and awkward:

- **No Maya dependency at all.** Not a soft dependency, not "works better if Maya is installed". It has to run on a build machine with no Autodesk software on it.
- **Never corrupt a file it writes.** Scene files are hours of artist work and there's no undo on a bad save.
- **Embeddable.** The same parsing code runs inside a standalone executable, a Houdini plugin built against the HDK, and a Maya command plugin. That pushed the design to header-only modules with no external libraries.

What falls out is a wider set of applications than the round trip that motivated it.

Headless batch auditing. One process walks a whole depot and emits a JSON record per file: mesh names, face and vertex counts, UV set names, shading assignments, cameras, file references, presence of specific custom attributes. Multithreaded across files with a shared work queue, so a deep reference tree is one process instead of one per node.

Shading that survives procedural work. Per-face shading partitions live in the file as component lists, so an unloaded reference can still say which shader owns which faces, which is what Maya won't tell you until it materializes the whole reference. Reading the partition before the geometry is fractured lets it ride through the fracture as an attribute and get reapplied on the way back. The alternative, matching shaders by proximity after the fact, is a guess.

Write-back without opening the DCC. Inserting a node, adding a dynamic attribute, authoring animation curves, removing a previous run's generated nodes, all as a structural edit to a file Maya wrote.

Interactive integration. The parser is header-only with no dependencies, so it drops straight into a Houdini SOP. A scene file becomes something you wire into a network and cook, with no import step and no intermediate format.

The rest of this paper is how the format actually looks, how I found that out, and how I made it safe to write.

---

3. The container as observed

Everything in this section is observed behavior across a corpus of real scenes, not documented behavior. I state it as fact where it held across every file I have tested and flag it where it did not.

3.1 Chunk anatomy

The `.mb` container is 64-bit IFF: big-endian, four-byte aligned, with two chunk kinds.

```
Leaf chunk:   [tag:4][flags:4][size:8][payload:size]
              header = 16 bytes

Group (FOR8): [tag:4][flags:4][size:8][subtype:4][children...]
              header = 16 bytes
              `size` covers subtype(4) + children
              children begin at offset+20
              group spans offset+16+size
```

Tags are four ASCII characters. Sizes are unsigned 64-bit big-endian, which is the "64" in IFF64 and why a chunk header is sixteen bytes instead of the classic eight. Groups always carry the tag `FOR8`, and what a group actually *is* lives in the four-byte subtype right after the header. A transform is `FOR8` with subtype `XFRM`, a mesh is `FOR8` with subtype `DMSH`. That matters more than it sounds like it should, and I come back to it in 3.3.

After every chunk the stream is padded with zero bytes to the next boundary, and the pad belongs to the enclosing region, not to the chunk before it. That distinction is invisible while you're only reading and becomes load-bearing the moment you write.

3.2 Alignment, and the four-byte correction

Maya `.mb` chunks are **four-byte aligned, not eight**. The eight-byte assumption is a natural one to make, since the size field is eight bytes and the container calls itself IFF64, and it is wrong.

It cost me a genuinely stupid week. A mesh group header sat at offset `0x145C`, four-byte aligned but not eight. My walker finished the previous chunk at `0x1464`, rounded up to `0x1468` under the eight-byte rule, and stepped straight over the `FOR8` tag. It did not crash. It did not warn. It fell through to a brute-force scan for the raw mesh payload and parsed the geometry correctly, so meshes came out with the right vertices, faces, and UVs, and without the node name, the vertex tweaks, the color set names, or anything else living as a sibling of the payload rather than inside it. The symptom was "some metadata is empty", which reads like a missing feature, not a parser walking off a cliff.

It bit a second time, months later, when one chunk-advance line still used `i % 8` after the rest had been corrected. Vertex tweaks were silently skipped for whichever assets had payload sizes that landed the next chunk differently, and those meshes loaded at the wrong local position because the raw vertex array was used without its tweaks. Same shape: partial correctness, no error, an offset that looks like somebody's export was bad.

Alignment is a whole-file invariant, and a parser that gets it wrong in one place produces a partial parse rather than a failure. If you correct an alignment rule, grep for every instance of the old rule in the same breath, because the one you miss will only reveal itself on files whose sizes happen to differ.

There is a further wrinkle. Maya's inter-chunk padding is not a clean closed-form rule across nesting contexts: sibling boundaries after the header group tend to land at multiples of eight, boundaries inside a transform group tend to land at four past a multiple of eight. Rather than model that, my production walker leans on two properties: **padding is always zero bytes, and a chunk tag is non-zero ASCII.** It advances to the declared end of the chunk, then skips zeros until it hits a printable byte, bounded by the enclosing region's end. The skipped pad is recorded as part of the chunk's source span, which is what makes byte-exact re-emission possible later.

The second of those is weaker than it sounds and I want to be exact about it. It holds for every stock chunk in my corpus, and plugin-authored chunks are the known exception: at least one uses a non-ASCII tag whose leading bytes are zero. A zero-skipping walker will run straight through such a tag and resynchronize somewhere arbitrary, which is the same silent-partial-parse failure as the alignment bug above. So the heuristic needs a bound and a bail rather than a loop: cap the skip at the maximum plausible pad, and treat overrunning that cap as a parse error at a known offset instead of scanning onward for something that looks like a tag. Skipping zeros is a recovery from not modelling the pad exactly, not a law of the format.

3.3 Nodes: the CREA chunk

Every node group opens with a `CREA` chunk that names it. The layout, verified across 13,316 `CREA` chunks with zero exceptions:

```
flags(1) + node_name\0 + [parent_path\0] + guid(16)
```

Bit 7 of the flags byte controls whether the parent string is present. Clear means two strings, set means one. The 16-byte identifier is always the last sixteen bytes and is never absent.

Flags	Meaning
`0x04`	Shape node with a parent
`0x05`	Default camera shape
`0x0c`	Transform with a parent
`0x80`	Standalone parentless DG node
`0x8c`	Root-level transform
`0x8d`	Default viewport transform

I first read that byte as a name-length code, because in the first file I looked at, `0x8C` sat in front of an eight-character name and the arithmetic worked. One file will support almost any theory. The census killed it: the correlation with name length evaporated across a wider corpus, and the correlation with parent presence held everywhere.

The byte layout matters less than what it implies about identity.

The node's type is never in the `CREA` chunk. It comes entirely from the enclosing `FOR8` subtype. A parser that infers type from the name or the payload is guessing; a parser that reads the subtype is reading the answer.

The DAG hierarchy exists only in the parent field. Nodes are flat siblings under the top-level scene group, and no chunk nesting mirrors the outliner. The parent field holds a short name when that name is unambiguous and a pipe-delimited absolute path (`|group|child|node`) when it is not, following Maya's minimal-path rule. Two consequences: a "subtree" is a set of flat siblings selected by resolved DAG path, not a nested region of the file, and keying anything on a bare short name is an identity trap, because scenes routinely contain dozens of nodes sharing a short name under different parents. I key on the resolved path and use short names only when the name is provably globally unique.

Transform payloads carry the usual channels plus rotate-pivot and scale-pivot, and their translate companions, as `DBL3` chunks. The pivot-aware composition I implement, in row-vector convention where a point is `v * M`:

```
no pivots:  M = S · R · T
pivots:     M = Sp⁻¹ · S · Sp · ST · Rp⁻¹ · R · Rp · RT · T
```

`ST` and `RT` are the scale-pivot and rotate-pivot translate terms. An earlier version of my builder dropped them and kept only the pivot and inverse-pivot pairs, which is correct until an asset actually uses a pivot translate, at which point the object lands somewhere plausible and wrong. The order above is the one validated against Maya's own world bounding box, and it's what ships.

It is still not Maya's complete local matrix. Maya also composes shear and a rotate axis, and joints add an orientation term. I don't model any of those, so a node using them resolves incorrectly and nothing in my code notices. Skipping pivots altogether is correct for objects whose pivots sit at the origin, which is most of them in a test scene and not enough of them in a production one.

3.4 Node types

Stock Maya node types appear as four-character ASCII subtypes. Twenty-six were confirmed across the census:

Code	Type	Code	Type
`XFRM`	transform	`PCTA`	animCurveTA
`DMSH`	mesh	`PCTL`	animCurveTL
`DCAM`	camera	`PCTU`	animCurveTU
`JOIN`	joint	`GPID`	groupId
`SHAD`	shadingEngine	`DSPL`	displayLayer
`DMTI`	materialInfo	`NTWK`	network
`RBLN`	blinn	`SCRP`	script
`RLAM`	lambert	`DPLM`	displayLayerManager
`RTFT`	file2DTexture	`OBST`	objectSet
`RLLK`	lightLinker	`PRTN`	partition
`RNLM`	renderLayer	`RNDL`	renderLayer (alternate)
`SDML`	shadingMap	`YNST`	instancer
`PSDM`	psdFileTex	`SLCT`	singleton (carries no `CREA`)

`Maya`, `HEAD`, and `CONN`/`CONS` are structural containers rather than node types.

Plugin-defined node types don't get an ASCII code. They carry a four-byte integer formtype, which reads as garbage if your dumper assumes printable tags. Worth knowing even if you never care about a specific plugin: a chunk walker that filters on "is this tag printable ASCII" silently skips every plugin node in the file, and the inventory it produces looks complete.

3.5 Mesh data

Geometry lives in a `MESH` chunk inside the mesh group.

```
Header (varies by type byte):
  Type 0x6f: data starts at offset 7, u32 vert_fc at offset 3
  Type 0x63: data starts at offset 8, u32 vert_fc at offset 4

Vertices:
  float32[vert_fc] big-endian        (vert_fc = num_verts * 3)

Edges:
  u32 edge_entry_count               (= num_edges * 2)
  for each edge: u32 a_raw, u32 b_raw   (mask 0x7FFFFFFF for the index)

Faces (edge-indexed):
  u32 face_entry_count
  for each entry: u32 raw
    edge_idx = raw & 0x1FFFFFFF      (29 bits)
    flags    = (raw >> 29) & 0x7     (3 bits)
    if flags & 0x4: use edges[edge_idx].second   (reversed direction)
    else:           use edges[edge_idx].first
    if flags & 0x3: end of face

UVs (after faces, still inside MESH):
  u32(0) + u32(uv_set_count) + u32(0)    12-byte preheader
  'mapN' + NUL                            uv set name
  u32 uv_fc                               (uv_fc = num_uvs * 2)
  float32[uv_fc] big-endian               (u,v pairs)
  u32 uvi_count
  u32[uvi_count] uv_indices               per face-vertex, in face order
```

Faces are edge-indexed, not vertex-indexed, with the direction bit selecting which end of the edge to take. Maya's winding is opposite to Houdini's, so building polygons downstream means reversing per face, and the UV indices have to reverse in lockstep or the texture coordinates shear by one vertex. Normals aren't in the chunk at all, they're computed, so any tool claiming to round-trip normals through this path is recomputing them and hoping the smoothing rules match.

3.6 Connections

The dependency graph sits near the end of the file, inside a `LIS8` container with subtype `CONS`, wrapping a `FOR8` group with subtype `CONN`. Each edge is a `CWFL` chunk with a compact payload:

```
\0 src_node.src_attr \0 dst_node.dst_attr \0
```

A leading null byte, then two null-terminated strings. The `CWFL` flags field distinguishes standard connections (`0x00`) from per-face or dynamic ones (`0x01`) and animation outputs (`0x02`).

Common patterns, from a scan of 12,497 connections:

Pattern	Meaning
`.iog -> .dsm`	Whole-object shading group membership
`.iog.og[N] -> .dsm`	Per-face group shading group membership
`.ciog.cog[N] -> .dsm`	Component instanced object group
`.oc -> .ss`	Material to shading group surface shader
`.id -> .iog.og[N].gid`	Group id linkage
`.mwc -> .iog.og[N].gco`	Wireframe color override (not shading, skip it)

Most of the metadata worth having names its nodes by string, so it resolves through this graph. I parse `CONN` before anything that depends on node identity.

3.7 Animation curves, and times that read as zero

Animation curves are `FOR8` groups with one of three subtypes, mapping directly to Maya's curve types:

Subtype	Maya type	Used for
`PCTA`	animCurveTA	Angles (rotation channels)
`PCTU`	animCurveTU	Float, unclamped
`PCTL`	animCurveTL	Float, clamped

Each curve group holds a `CREA` naming it, two `DBLE` leaves for the curve-level tangent type and weight, and then a run of `FLGS` + `CMPD` pairs, one pair per per-key array: `ktv` (the keys themselves), `kbd`, `kyts`, `kit`, `kot`, `ktl`, `kwl`, `kix`, `kiy`, `kox`, `koy`.

The `FLGS` chunk that precedes each array declares its length:

```
attr_name\0 + 0x28 + u32_be(array_count)
```

The `ktv` payload is the attribute name, a null, a `0x20` separator, and then sixteen bytes per key:

```
int64_be(maya_ticks) + float64_be(value)
```

Maya's tick rate is 141,120,000 ticks per second, constant across versions. Frame number is `ticks / (141120000 / fps)`, so one frame is 4,704,000 ticks at 30 fps and 5,880,000 at 24 fps.

That took me a week I didn't need to spend. My first decode read both halves of each key as IEEE 754 doubles, because a time and a value are both continuous quantities and that's the obvious guess. Every value came out correct. Every time came out as `0.0`. Not garbage, not a wild number, exactly zero, on every key of every curve in a file with thousands of hand-set keys.

A double's exponent field is the eleven bits directly under the sign bit, so reinterpreting an integer as a double puts that integer's high bits into the exponent. Any tick count below 2^52 leaves the exponent field entirely zero, which makes the result subnormal: frame 1 comes out as 2.3e-317, frame 10,000 as 2.3e-313. Every one of those prints as `0.0` at any sane precision. And 2^52 ticks is roughly 9.6e8 frames at 30 fps, so the degenerate case isn't an edge case, it's every frame number anyone will ever key. The wrong type didn't produce a wrong-looking answer. It produced a uniform degenerate one, which reads as missing data rather than misread data.

A field that decodes to a plausible but degenerate value under the wrong type is worse than one that crashes. I spent days looking for a reason the times weren't being written. They were written. I was asking them the wrong question. The signal I should have caught is uniformity, because real data is never that clean. If a decode returns the same value for every record in a large file, suspect the decode before the data.

One more encoding detail, because it looks like a bug the first time you hit it. A per-key tangent array whose `CMPD` body is exactly four zero bytes, while its `FLGS` declares a non-zero count, is not truncated. It's a sentinel: **every key uses the curve-level default tangent type** from the curve's `tan` value. Files with hand-authored tangents carry populated arrays of doubles instead. Treat the sentinel as malformed and you either reject good scenes or fabricate a zero tangent on every key.

3.8 Cameras

Camera shapes are `FOR8` groups with subtype `DCAM`, holding a flat dictionary of short-name attributes. The ones that carry real information:

Attr	Long name	Notes
`fl`	focalLength	mm
`coi`	centerOfInterest	Distance to look-at target
`ow` / `o`	orthographicWidth / orthographic	`o` = 1.0 selects ortho
`cap` / `hfa` / `vfa`	filmAperture	Inches. Default 36mm x 24mm if absent
`ncp` / `fcp`	near/farClipPlane	Default 0.1 / 10000 if absent
`hfv` / `vfv`	h/vFieldOfView	Not stored when `fl` is used

The rest are the flags and display settings you'd expect: renderability, visibility, film fit, overscan, focus distance, gate mask opacity and color, resolution gate, per-pass image names, and a MEL hot command for viewport binding.

Two things bite. Film aperture is stored in inches while focal length is in millimetres, so any mapping into another application's camera carries a `* 25.4`. And absent means default, not zero, so a camera that never had its clip planes touched has no clip plane chunks at all and a parser that reports 0.0 for them has invented a camera that clips everything.

3.9 What a real scene looks like

Group census from one production-scale animation scene, roughly 50 MB, two animated characters and six cameras:

Group	Count	Purpose
`PCTU`	392	Animation curve, float unclamped
`PCTL`	224	Animation curve, float clamped
`PCTA`	223	Animation curve, angle
`XFRM`	87	Transforms
`GPID`	73	Group ids
`DMSH`	72	Meshes
`DMTI`	47	Material info
`SHAD`	47	Shading groups
`DCAM`	6	Camera shapes
`CONN`	1	Connection graph
`JOIN`	1	Skeleton hierarchy

That table is the argument for decoding animation at all. Curves are 839 of the 1173 groups in that census, two and a half times every other kind of node combined and nearly ten times the transform count, so a scanner that reads meshes and stops has read a small minority of a character scene.

---

4. Methodology

The method transfers better than the format does. This is the process, in the order it runs.

4.1 Differential analysis, with the application as the ground truth

The highest-yield technique by a wide margin: save a controlled scene, change exactly one thing, save it again, diff the two chunk trees. Whatever appears in the second tree and not the first is that one thing's on-disk encoding, isolated.

That's how I got the dynamic-attribute encoding. A vanilla transform node and the same node carrying one added boolean attribute differ by exactly two leaf chunks: an `ATTR` chunk holding the definition,

right after the node's `CREA`, and a value chunk appended at the node's end. No guessing about layout, no theory about how Maya packs attribute metadata. The diff is the template, and I lift the real bytes out of it instead of synthesizing my own.

The discipline that makes it work is keeping the control tight. One change per save. If the diff shows more than you expected, the scene changed more than you thought, and reasoning from a noisy diff is how you end up with a format fact that's really an artifact of your test.

4.2 Python for discovery, C++ for production

Discovery code and production code have opposite requirements. Writing one piece of code that does both makes both worse.

Discovery wants to be wrong cheaply. It prints everything, tolerates malformed guesses, restarts from a byte offset I typed by hand, and gets thrown away. That's Python. `_dump_all_chunks.py` walks a group recursively and prints tag, offset, size, an attribute-name hint from the payload's first null-terminated string, and a hex preview. `_hexdump_range.py` is fifteen lines dumping a byte range with an ASCII gutter, and I've used it more than anything else in the set. Two habits across the rest: each probe states its hypothesis in the docstring and implements exactly that, so a failed run falsifies something specific, and several exist only to find files that exercise a rare branch, because most scenes hit the common case and prove nothing.

Production wants to be right cheaply. Header-only C++ with no external dependencies, so the same parser compiles into a standalone scanner, a Houdini plugin, and a Maya command plugin with no build-system negotiation. One header per format area.

The bridge is deliberate. A Python probe establishes what the bytes mean, a C++ header implements it for real, a standalone `_test_*.cpp` driver proves the header still does what the probe found. Every parser header has one, they print findings in a form I read in five seconds, and they run without a DCC. `_mb_diff.cpp` walks two trees in lockstep and dumps added or changed chunks with hex; `_test_mb_roundtrip.cpp` is the writer's base invariant and gets its own section below.

4.3 Re-opening in Maya is the only real test of a write

For reading I can check my answer against Maya's own queries. For writing there's one authority, which is whether Maya opens the file. A file can parse cleanly under my reader, round-trip through my serializer, and still be rejected, because my reader is a model of the format and Maya is the format.

That test is binary and unhelpful. A structurally broken `.mb` produces one line, "Error reading file", with no offset and no clue. So the test has to be cheap and pass/fail: write, open, look. Anything that slows that loop slows the whole project, because it's the loop I run hundreds of times.

4.4 Corpus discipline

A parse rule is a hypothesis until it survives a census. My working corpus is four files from a few megabytes to roughly 850 MB, and a rule gets promoted to confirmed only when it holds across all of it. That's where the counts in this paper come from: 26 ASCII formtypes confirmed, plus an unenumerated set of plugin-authored formtypes that are integer-coded rather than ASCII, 12,497 connections scanned, and 13,316 `CREA` chunks parsed with zero exceptions.

The size spread matters as much as the count. Small files exercise defaults and empty cases. Large files exercise the branches that only appear at scale, including the ones where a size field crosses a threshold no test scene will reach.

4.5 Research logs with visible unknowns

Every format area gets a research note with an open-questions list at the bottom. When something is solved the entry stays and gets struck through with the answer inline. Nothing is deleted.

That sounds like bookkeeping. It's a correctness tool. The failure mode in this work isn't forgetting an unknown, it's an unknown quietly graduating into an assumption. Six weeks after I write "not sure what this 4-byte field is", the code around it looks confident and reads as though somebody decided. A visible open-questions list keeps the boundary between "I verified this" and "this has held so far" intact. Section 9 is lifted straight from those lists.

---

5. Reading without a spec: a failure catalog

Five real failures. Nothing crashed in any of them. Each one produced a confident wrong answer, or a confident wrong impression, and kept going.

5.1 Counts of what

Vertex counts came out three times too high on every mesh in every file, and a caching layer that trusted the number went into rebuild loops and scan-failure fallbacks.

The count was exactly 3x, so the field stores floats, not vertices. The `u32` after the mesh separator is a float count, `num_verts * 3`. My parser read it as a vertex count because it sits where a vertex count would sit and has roughly the magnitude one would have.

Integer fields in this format carry a semantic role, count of what, that the layout doesn't state. The same four bytes could be vertices, floats, or bytes, and the file won't say. My heuristic now is arithmetic: exactly 3x wrong is floats in a vec3 stream, exactly 2x is a UV pair stream or an edge endpoint stream, exactly 4x is bytes. The edge and UV arrays in section 3.5 follow the same convention.

5.2 A field I decided was lying, which wasn't

Face-to-shader assignment was the blocker for reading shading out of unloaded references. The connection graph told me which object group on a mesh wired to which shading group. It didn't tell me which faces belonged to which object group.

The component list turned up as a `CMP#` chunk with a nested `CMDF` sub-chunk, and each component entry, the `f[a:b]` or `f[x]` from Maya's own component syntax, is a big-endian `u32` pair of start and end face index. The corrected layout:

```
[attr_name\0][0x20][be32 = 1]["CMDF"][be32 P][P x (be32 start, be32 end)]
```

The `be32` after `CMDF` is **P, the count of pairs**. The `be32` ahead of it is a leading frame count, invariantly 1 in every chunk I've seen.

I read that pair count as a byte size. Every size-shaped field I'd decoded up to that point was a byte count, so I pattern-matched, and the number came out far too small for the stream that followed. One chunk declared 216 where the pair data ran on for 1728 bytes. I decided the field was truncated, wrote it up as a size field that lies, and built a rule on the disagreement: distrust declared sizes, trust the parent's boundary.

The field was a count and it agreed perfectly. 1728 bytes at eight bytes a pair is 216 pairs, exactly what the chunk said. There was never a disagreement. I invented one by deciding what unit the number was in, then explained my own error as a quirk of Maya's serializer and moved on. It took a later pass, byte-verifying re-encodes against Maya, to see the number I'd called a lie had been right all along.

A field's units are part of its layout, and nothing in a binary format announces them. I generalized "size-shaped fields here are byte counts" from a handful of samples and applied it to a field counting something else. The check I should have run costs nothing: if the value times a plausible element width lands exactly on the stream length, it's an element count, not a byte size.

Defensive parsing survived the wrong theory, which is why nothing shipped broken. The parser reads pairs until the enclosing `CMP#` payload ends, bounded by the parent. I wrote that for the wrong reason and it's correct anyway, because P and the parent boundary agree. Bounding a read by its container is right whether or not you understand the field that also describes it, and it kept a wrong explanation from becoming wrong data.

5.3 Untrusted sizes are untrusted, including your own

The chunk walker read `data_size` from the file and used it to allocate and seek, so a chunk declaring a multi-gigabyte size would either fail to allocate or read past the end of the buffer. The fix is one line, bounds-checking `data_size` against the bytes remaining in the enclosing region. What surprised me is where that check actually fires. "Malformed file" is the wrong mental model: in a reverse-engineered

parser the common source of an absurd size is **my own walker being a few bytes out of position**, reading a size field out of the middle of a float array, which happens constantly during discovery by design. So the bounds check isn't hostile-input defense, it converts a silent runaway or a crash three functions later into a local error at the exact chunk where the walk went wrong. It made the discovery loop faster, which isn't what I expected from a robustness fix.

5.4 The fallback that hid the working code

Meshes with multiple UV sets round-tripped correctly for set 0 and lost everything after it. The multi-set code was fine. An older, narrower fallback path, written as a guess early on and never removed, ran first, handled set 0, emitted a warning, and returned success. The correct parser downstream never executed on a real file.

The warning had been there for months. I'd stopped seeing it.

The fix was deletion. The fallback wasn't repaired or reordered or gated behind a flag, it was removed outright, and the in-chunk parser became the only path. That's the right ending for a guess that got superseded: once the real decode works, the guess isn't a safety net, it's a second implementation competing for the same input and winning by being first.

When you're debugging silent data loss, look for paths that silently succeed. A fallback returning partial data with a warning is worse than one that fails, because it removes the pressure that would have made someone look.

5.5 The fast path that died quietly

A different shape of the same problem, and the more expensive one. A fast native path had a fallback chain behind it: if it couldn't run, execution dropped to a path that was fully correct and far slower. Several unrelated conditions could trigger the drop and none of them announced themselves.

So when the fast path stopped loading, nothing reported an error. The work completed and the results were right. The only symptom was that everything took several times longer, which arrives as "the tool feels slow" rather than "a component failed to load", and that misdirects the diagnosis completely. Time goes into profiling the slow path and rewriting code that was never the problem, while the cause is one failed load that nothing printed.

A fallback to a slow but correct path hides its own death. Correctness-preserving fallbacks are a server-side pattern, where the caller is a script and the goal is never to throw. In a tool with a person in front of it the tradeoff inverts: they've already invested setup time, and silently spending it at a fraction of the speed teaches them the tool is slow rather than that something broke. The fast path is required now. If it can't run, it raises with a diagnostic naming the specific reason, and the slow path is available only as a conscious opt-in for the rare machine that genuinely needs it.

---

6. Writing without a spec: verbatim unless dirty

Reading a format you only partly understand is a research problem. Writing one is a risk problem, and the risk isn't evenly distributed. A wrong parse costs me an hour. A wrong write costs an artist a day of work with no undo, and it might not surface until they open the file next week.

A writer that regenerates the whole file has to be right about every chunk in it, including the ones it doesn't understand. I understand a good fraction of this format. I don't understand all of it and I never will, so any design that requires me to is going to eat somebody's scene.

6.1 The architecture

Every parsed chunk records its exact source byte span: header, plus payload or children, plus the trailing pad. Along with a dirty flag.

```
struct Chunk {
    char     tag[4];
    uint32_t flags;
    bool     is_group;
    char     subtype[4];              // groups only
    std::vector<uint8_t> payload;     // leaf only
    std::vector<Chunk>   children;    // group only

    const uint8_t* raw_ptr = nullptr; // verbatim source span
    size_t         raw_len = 0;
    bool           dirty   = false;
    size_t         pad_len = 0;       // original trailing pad
};
```

Serialization is then a two-case function. If the chunk is clean, copy its source span out byte for byte and return. If it is dirty, rebuild the header, write the body from structured fields, backpatch the eight-byte big-endian size, and replay the pad.

```
static void serialize_chunk(const Chunk& c, std::vector<uint8_t>& out) {
    if (!c.dirty && c.raw_ptr && c.raw_len) {
        out.insert(out.end(), c.raw_ptr, c.raw_ptr + c.raw_len);
        return;                       // untouched data is copied, never regenerated
    }
    // ... rebuild header, emit body, backpatch size, replay pad
}
```

An edit marks its chunk dirty and walks that flag up the ancestor chain, because every enclosing group's size field changes. Everything off that path stays clean and gets copied.

Four properties fall out, and they're the reason for the design:

1. **The identity round-trip is byte-perfect by construction.** Nothing dirty means everything verbatim means output equals input.
2. **A mutation's serializer surface is limited to the edited path.** Inserting one attribute into one node makes my serializer responsible for that node's group and its ancestors, and nothing else in a 50 MB file.
3. **Maya's exact flags and padding on untouched data survive by copy.** I don't have to know why a chunk has the flags it has.
4. **Undecoded chunks are safe.** The parts of the format I haven't figured out pass through untouched, because untouched is the default and not a special case.

The fourth one is why writing to an undocumented format is defensible at all.

6.2 The base invariant

Before any mutation code existed, the round-trip test did this:

```
read file -> build chunk tree -> serialize -> byte-compare against source
```

`_test_mb_roundtrip.cpp` runs it in memory, with no disk write by default, so it's safe against read-only paths. PASS means the chunk-tree model and the alignment handling are faithful. FAIL prints the offset of the first differing byte, which points straight at what the model got wrong.

That test is the license to write. A serializer that can't reproduce a file it didn't change has no business changing one. Every new file shape that enters the corpus goes through it before anything else.

6.3 The one that got through

The mutation path is: splice the template chunks for a dynamic attribute into the target node's group, then bump a counter in the file's trailing header group. That counter, `OBJN`, is a plug-id high-water mark, and it's stored as **ASCII digits**. That last detail is what breaks.

The serializer replayed each dirty chunk's captured pad, because Maya's padding is irregular and a recomputed four-byte pad under-pads in some contexts, shifting every downstream chunk. Replaying the original pad is correct in general. It's correct for `OBJN` too, **as long as the digit width doesn't change.** `4271` becomes `4288`, four ASCII bytes stay four ASCII bytes, the pad is still right, the file is fine.

Cross a digit-width boundary, `9999` to `10000`, and the payload grows by one byte while the pad stays the length it was. The `OBJN` chunk is now one byte wider than its aligned slot. Every chunk after it in the file is misaligned by one. Maya rejects the entire scene with "Error reading file", no offset, no node name, nothing.

Three things about this bug matter more than the fix.

It was latent in a shipped path. Not new code. The write path had been in use and correct the whole time, because nothing in production had crossed a digit-width boundary yet. The bug needed a specific magnitude to fire, so it sat there harmless until it wouldn't have been.

It only shows up as total loss. There's no partial version. The file is fine or it doesn't open, which is the good case, oddly: the artist finds out immediately instead of from a subtly wrong scene three weeks later.

The fix had to preserve the existing behavior exactly. The payload and its pad together form an **aligned slot**. Keep the slot when the wider value still fits, grow it by the local alignment period only when it doesn't. That period is 8 here rather than the file-wide 4, because `OBJN` sits in the trailing `HEAD` group, whose sibling boundaries are the mod-8 case from 3.2. Growing by the period the surrounding region actually uses is the point; the four-byte rule still governs the container.

```
size_t old_slot = leaf.payload.size() + leaf.pad_len;
leaf.payload.assign(ns.begin(), ns.end());
size_t new_slot = old_slot;
while (new_slot < leaf.payload.size()) new_slot += 8;
leaf.pad_len = new_slot - leaf.payload.size();
leaf.dirty = true;
```

The slot is unchanged whenever the digit width is unchanged, so **output is byte-identical to the old behavior on every case the old behavior handled**. The new path is a superset of the shipped one. That's not an aesthetic preference. It's what let me ship a fix to a corruption bug in a file writer without re-validating every scene the old path had ever produced, and I'd apply it to any change in a serializer that already has files in the wild.

Verification was direct. The pre-fix build corrupts a file on a width crossing and Maya fails the open, the fixed build produces a file Maya opens clean. Both halves matter. A test that only proves the fixed build works doesn't prove the test can detect the bug.

Replaying captured bytes verbatim is only correct while the field those bytes wrap keeps its size. Any writer that edits a variable-width payload in place has to recompute alignment, not replay it. Replay-based serialization is a good technique and this is its sharp edge, so enumerate every field you mutate whose serialized width can change and treat each one as an alignment problem, not a value problem.

6.4 Store and replay for payloads you cannot regenerate

Some payloads carry fields I can't reproduce. One attribute I decode mixes a few fields I never decoded, one of which behaves like a checksum, with a length field and a readable JSON body.

I can read the JSON and I could rewrite it. I can't produce a matching value for the checksum-shaped field, and I don't know whether the consumer validates it.

So I don't guess and I don't strip it. I **store the entire payload as raw bytes on read and write back exactly those bytes**, extracting the readable portion separately for inspection. Modification is refused until that field is understood. That's verbatim-unless-dirty applied at payload granularity instead of chunk granularity, and it means a tool editing other parts of the file carries this payload through without understanding it.

The length field in that same payload cost me time twice over.

It's little-endian, inside an otherwise uniformly big-endian container. The container is big-endian because IFF is big-endian. This payload is written by a dynamic-attribute serializer using native host order, and the host is x86. **Container endianness is not payload endianness.** A nested serializer carries its own conventions, and you catch it by noticing that a length decodes to an absurd number one way and a sensible one the other.

And it's C-string style, counting the trailing NUL. A length of 200 means 199 bytes of text. Get it wrong and every extracted string carries a stray null that fails to compare equal against the same string from anywhere else, invisibly in a log.

6.5 Removal and the identity trap

Removal uses the same architecture. Erase the child chunks from their group, mark the group and every ancestor dirty, scrub the matching connection edges out of the connection container, audit for dangling references before writing. Survivors emit from their captured spans, only the edited containers rebuild.

Section 3.3's trap bites hardest here. Removing a subtree means selecting flat siblings by resolved DAG path, and generated wrapper nodes frequently all carry the identical short name, legal as long as each is unique under its own parent. **Any removal keyed on a bare short name hits every one of them in the scene.** Removal and the dangling-reference audit both key on the resolved full path.

---

7. Validation

Speed numbers are the least interesting evidence a tool like this can offer. A fast parser that's wrong is worse than no parser. These are the checks I trust.

7.1 Count cross-matching between independent decoders

This is the strongest evidence in the project.

The connection-graph decoder walks a container near the end of the file and parses `CWFL` edges. On one production-scale animation scene it found **1394 connections**, **839** of them with an animation curve's output as the source attribute.

The animation-curve decoder walks a different part of the file looking for `FOR8` groups with the three curve subtypes. It found **839 curves**.

Two decoders, two chunk families, two regions of the file, no shared code and no shared assumption, one number. That agreement rules out the systematic failures. A curve walker that missed a nesting context under-counts. One that double-counts from a mis-seek over-counts. A connection parser that mis-handles the leading null byte or the attribute suffix gets the wrong subset. Any of those and the numbers don't meet. They met exactly.

Build cross-checks between paths that share nothing. Two agreeing implementations of the same walk prove the walk is deterministic. Two agreeing counts of the same entity reached from opposite ends of the file prove the count is right.

7.2 Domain invariants as single-bit tests

All 839 curves decoded to keyframe times on **integer frames at 30 fps, 100% of them**.

That's stronger than it looks. The tick decode has several independent ways to be wrong: the rate constant, the endianness, the signedness, the per-key stride, the offset of the time field inside the key. Get any of them wrong and the output isn't slightly off, it's a spray of fractional frames with no pattern. Animators key on frames, so the domain guarantees the property the decode has to reproduce, and a broken decode doesn't produce it by accident.

Look for invariants like that in whatever you're decoding. One boolean over the whole dataset beats a hundred spot checks.

7.3 Structural counts that expose ambiguity

The transform pass on the same scene reported **87 transform records, 52 unique names, 87 unique DAG paths**.

The discrepancy is the finding. That scene has two characters with identical joint naming, so 35 short names collide. A name-keyed map keeps the last one of each and loses 35 of them. A path-keyed map keeps all 87. I only knew to build the path-keyed map because I counted both.

Count your entities more than one way. The gap between two counts of the same thing is where the identity model is wrong.

7.4 Bit-for-bit comparison against Maya's own output

Where Maya can answer the same question I diff against it rather than sanity-checking against it. The per-face partition decode was compared per shading group and per face count against the identical live query. One two-material mesh resolved to 1736 faces in one group and 616 in the other, matching Maya exactly, not approximately.

Structural rules get the same treatment corpus-wide. The `CREA` layout held across 13,316 chunks with zero exceptions, but those chunks came from four files, so what I can honestly claim is a rule that survived every instance in a four-file corpus, not a rule verified against the format. The distinction matters for exactly the reason section 5.2 exists.

7.5 What the re-open test catches

For writing the pipeline is identity round-trip byte-compare, mutate, serialize, open in Maya. Only the last step is authoritative.

Corruption class	Cause	Signature
Whole-file misalignment	Pad replayed for a payload that changed width	Maya refuses the file, one error line, no offset
Downstream shift	Size field not backpatched after an edit	Same, or a truncated node tree
Dropped edit	An ancestor left clean, so its verbatim span overwrites the change	File opens fine, edit silently absent
Rejected payload	Body modified under a checksum that was not regenerated	Consumer rejects or reverts the value
Identity-trap deletion	Removal keyed on a non-unique short name	Nodes disappear from unrelated parts of the scene

The third row is the one to watch in any replay-based serializer. The failure isn't corruption, it's a no-op, and a no-op looks exactly like a tool that didn't run.

---

8. Performance

These are measurements from a working pipeline, not a benchmark suite. Each is one scene, and the ratios vary with piece count, frame count, mesh complexity, and disk. Take the order of magnitude and the reason for it, not the digits.

Native export against the Python implementation it replaced. On a 951-piece, 86-frame packed cache: 305 seconds down to 8.2 seconds, roughly **37x** on that scene. The multithreaded build is faster still and the exact ratio varies by cache.

Scan cost per file. A reduced scan mode that skips hierarchy resolution, shading resolution, and a second decoding pass cuts per-file cost roughly **3-4x**, for scans that only need node names and reference paths. Full output needs the full pass.

Keyframe application on the Maya side, same 951-piece, 86-frame scene, applying all channels:

Path	Time	Relative
`cmds.setKeyframe`	142 s	1x
Python API (`MFnAnimCurve`)	18 s	~8x
C++ command plugin	2.8 s	~50x

Where the time goes matters more than the ratios. Most of it is **startup and licensing**, which for a batch job is the dominant cost of the conventional approach and a fixed one no optimization inside the DCC can touch. **One process amortizes I/O**, so a reference tree scan keeps the OS cache warm and the workers busy instead of paying setup per file. **A targeted scan reads only what I need**, walking to the chunks it wants, while loading a scene materializes all of it, references included. And per-file work parallelizes cleanly because the files are independent, where the equivalent work inside a DCC is serialized behind a single-threaded evaluation.

---

9. Limitations and open chunks

Everything here is either not decoded, decoded but under-tested, or a standing risk I've chosen to live with.

9.1 Fields I can see but can't explain

A handful of 4-byte fields inside one attribute payload are still undecoded. One holds the same value across every record in a file. One varies per record and behaves like a checksum, which is why that payload is under store-and-replay per section 6.4. One is a constant byte pattern in every instance I've looked at.

I write them up that way, by observed behavior, rather than calling them reserved. "Reserved" is a claim about intent I have no basis for, and anyone extending this work is better served starting from what the bytes actually do than from a guess of mine that reads like a finding. Section 5.2 is what happens when I forget that.

9.2 Group types parsed but not decoded

Skeleton hierarchy groups (`JOIN`) and group-id records (`GPID`) are walked and preserved, not interpreted. Joint hierarchies and set membership don't round-trip through my tooling, they only

survive it. For a destruction pipeline that's an acceptable gap. For character work it's the first thing to fix.

9.3 Things I need from outside the file

Frames-per-second is the clearest case. Keyframe times are absolute ticks, so converting to frames needs a rate the curve data doesn't carry. I take it from a manifest or from the caller. Getting it wrong produces plausible animation running at the wrong speed, which is the worst kind of wrong.

9.4 Branches decoded but thinly exercised

Per-key tangent arrays are implemented and correct as far as I can tell, but most production scenes hit the default-tangent sentinel instead of populating them, so the populated branch runs against a handful of files. Both mesh header type bytes are handled, but one is far more common than the other in my corpus. Undertested isn't the same as broken, and it isn't the same as tested either. Normals are a related gap. They aren't stored in the mesh chunk at all, so anything I produce is a reimplementation of somebody else's smoothing rules and matches only as far as that reimplementation is faithful.

9.5 Scope of the writer

Gestir does **structural read-modify-write**, not scene serialization. It inserts, removes, and edits chunks in a file Maya wrote. It isn't a general `.mb` authoring library, and that distinction isn't modesty. Every safety property in section 6 depends on having a real Maya-written file underneath to copy the untouched parts from. Writing a scene from nothing removes the verbatim default and puts me back to needing to be right about every chunk, which is what the whole design exists to avoid.

The writer also inherits the identity problem from 3.3 rather than solving it. Duplicate short names are legal and common, so every lookup that keys on one is lossy and the DAG-path map is the only authoritative index. I route around that everywhere it matters, which is not the same as fixing it: any consumer I hand this to that keys on a short name is a latent bug waiting for a scene with two of something.

9.6 Version drift, and what this isn't

Everything here is observed behavior of the files in front of me, from the Maya versions I have. A future release can change a field with no notice, no error, and no version bump I can see. My only detector is a failing invariant, which is why the tests in section 7 aren't optional and why the identity round-trip runs against every new file shape that enters the corpus. That's the standing cost of building on a reverse-engineered format and it doesn't go away.

The other boundary is evaluation. Gestir reads storage, it doesn't evaluate. Constraints, deformers, expressions, and plugin node behavior are all evaluation, and none of them are answered by reading the file. When the answer depends on what Maya computes rather than what Maya stored, opening the

scene is still the right call. Pretending otherwise is the kind of overreach that makes a tool untrustworthy.

---

10. Future work

In-DCC cook integration. Wrapping the same headers as a node that cooks inside a Houdini session removes the subprocess boundary the write path currently crosses, and makes write-back happen mid-network instead of as a separate export step.

Batch audit as a callable module. The scanner is fastest when one process handles many files, and it's currently reached through a command line. As an importable module a whole-depot audit could run in-process and skip the JSON round-trip entirely.

Decode `JOIN` and `GPID`, the two remaining structural gaps and the ones standing between this and useful character-scene tooling.

Decode the checksum field. Turning store-and-replay into safe editing is a contained, well-defined problem, and the payoff is that a whole class of attribute becomes writable instead of merely preservable.

A synthetic test corpus. The largest gap, and the thing I'd build first if I started again. Every test here runs against real scenes, so coverage is whatever those scenes happen to contain, and the branches that matter most are the ones real scenes rarely exercise: a curve with fully populated tangent arrays, a counter one increment below a digit-width boundary, a mesh using the less common header type byte, deliberately colliding short names. Generated scenes hitting each branch on purpose would turn the invariant checks into a regression suite independent of any particular file, and make this a reproducible artifact instead of a set of findings.

That's the honest summary of where the work stands. The format findings are solid, the architecture has held up in daily use, and the test story still rests on the corpus I happened to have.

Errata

Corrections to this paper are appended here with dates, rather than silently edited into the text above. A format finding that turns out to be wrong is worth more as a dated correction than as a quiet overwrite, which is the whole argument of section 5.2.

No corrections yet.

Gestir is written and maintained by Rigel Bowen as the native layer of Yggdrasil, a Houdini-Maya destruction round-trip pipeline. No part of it links against or redistributes Autodesk code.